Librería matplotlib

M4: Análisis exploratorio de datos

|**AE6**: Aplicar personalizaciones a los gráficos de datos utilizando la librería matplotlib para la resolución de un problema dado.

alkemy

Introducción👩‍💻

La visualización de datos es una de las habilidades más esenciales en el análisis de datos, ya que permite representar gráficamente información compleja de forma clara, comprensible y atractiva. En este contexto, la librería **Matplotlib** se posiciona como una de las herramientas más robustas y versátiles dentro del ecosistema Python. Permite generar gráficos de calidad profesional, con amplias posibilidades de personalización, desde simples líneas hasta representaciones más complejas como diagramas de dispersión o boxplots.

Este manual está diseñado para guiarte paso a paso en el dominio de los principales aspectos de Matplotlib, desde su estructura básica hasta las técnicas avanzadas de personalización de gráficos, abordando cada tema con profundidad y con ejemplos claros que te permitirán aplicar lo aprendido a situaciones reales.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Aplicar personalizaciones a los gráficos de datos utilizando la librería Matplotlib para la resolución de un problema dado.
- Identificar las principales características de la librería Matplotlib para el análisis visual.
- Seleccionar el tipo de gráfico adecuado según la naturaleza del conjunto de datos.
- Construir gráficos visualmente efectivos y personalizados para representar diferentes tipos de información.

Características de la librería Matplotlib

Matplotlib es una librería de visualización bidimensional escrita en Python. Fue desarrollada con la intención de replicar la funcionalidad de MATLAB en cuanto a gráficos, pero con la ventaja de ser completamente open source. Lo que la hace especialmente poderosa es su capacidad de generar gráficos con un alto grado de personalización, además de permitir su integración con otras bibliotecas como NumPy, Pandas y Seaborn.

A diferencia de otras herramientas, Matplotlib puede utilizarse tanto desde un enfoque orientado a scripts simples como desde una estructura más robusta basada en objetos. Esto la convierte en una opción apta tanto para principiantes como para desarrolladores avanzados que requieren un control más fino sobre cada componente del gráfico.

Una de sus principales ventajas es que permite exportar los gráficos en múltiples formatos de alta resolución (como PNG, PDF, SVG, entre otros), siendo útil no solo para notebooks interactivos sino también para presentaciones y publicaciones.

Importación de la librería

Para comenzar a trabajar con Matplotlib, es necesario importar el módulo pyplot, que agrupa la mayoría de las funciones necesarias para crear gráficos. La convención más habitual es hacerlo como plt para simplificar el código:

```
import matplotlib.pyplot as plt
```

Este módulo funciona como una interfaz de estilo procedural, similar a MATLAB. Si bien existe un enfoque más estructurado basado en clases, pyplot ofrece una manera más rápida y sencilla de comenzar a graficar. En la mayoría de los casos se complementa con NumPy para generar o manipular datos numéricos.

Componentes principales de un gráfico

Cada gráfico de Matplotlib está compuesto por una estructura jerárquica de objetos. En la base de esta jerarquía se encuentra la **Figure**, que actúa como el lienzo general sobre el cual se dibujan los distintos elementos. Dentro de la figura se encuentran los **Axes**, que son áreas individuales de graficado, donde se colocan las líneas, barras, puntos u otros elementos.

Cada Axes contiene sus propios ejes (X e Y), etiquetas, ticks, leyendas, títulos y otros componentes gráficos. Esta separación permite una gran flexibilidad, ya que una misma figura puede contener múltiples áreas de graficado independientes.

El modelo de Matplotlib se puede representar de la siguiente manera:

Componente	Función
Figure	Contenedor principal del gráfico
Axes	Área de dibujo con sus propios ejes
Axis	Líneas que representan los ejes X y Y
Ticks	Marcas numéricas en los ejes
Labels	Nombres de los ejes
Legend	Descripción de las series representadas
Title	Título general del gráfico

Crear un gráfico con las funciones pyplot

El enfoque basado en pyplot es ideal para tareas rápidas. Las funciones se ejecutan de manera secuencial, lo que hace que el flujo de trabajo sea muy directo. Por ejemplo, si se quiere representar la evolución de una variable a lo largo del tiempo, se puede hacer con unas pocas líneas de código:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 20, 18]
plt.plot(x, y)
plt.title("Evolución temporal")
plt.xlabel("Tiempo")
plt.ylabel("Valor")
plt.show()
```

Este método es muy útil para análisis exploratorios o presentaciones simples. Sin embargo, para casos más complejos se recomienda usar el modelo orientado a objetos.

Crear un gráfico con modelo de orientación a objetos

Este enfoque permite mayor modularidad y reutilización de código. Es especialmente útil cuando se quieren crear múltiples subgráficos dentro de una misma figura o cuando se necesita manipular de forma independiente varios elementos del gráfico.

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("Gráfico OO")
ax.set_xlabel("Tiempo")
ax.set_ylabel("Valor")
plt.show()
```

Ambos métodos pueden generar los mismos resultados visuales, pero el modelo orientado a objetos es más adecuado cuando se necesita personalizar cada parte del gráfico o incorporar múltiples elementos interactuando entre sí.

Figuras y subgráficos

Una de las funcionalidades más potentes de Matplotlib es la posibilidad de generar varios gráficos en una misma figura. Esto se logra usando la función subplots, la cual permite dividir el área total en múltiples secciones.

Por ejemplo, para mostrar dos tipos de gráficos uno al lado del otro, se puede usar:

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))
axs[0].plot(x, y)
axs[1].bar(x, y)
```

La posibilidad de compartir ejes (sharex, sharey) facilita la comparación entre los gráficos, y el uso del parámetro figsize permite definir dimensiones acordes al espacio de presentación.

Posición y tamaño de los gráficos

El tamaño de una figura no solo influye en su legibilidad, sino también en su adecuación a diferentes medios. Se puede definir desde la creación de la figura con el parámetro figsize en plt.figure() o plt.subplots().

Además, si es necesario ajustar la distribución del espacio entre subgráficos, se puede utilizar plt.tight_layout() o plt.subplots_adjust() para evitar superposición de etiquetas o ejes.

Colores, marcadores y estilos

Personalizar los elementos visuales del gráfico es fundamental para comunicar claramente los datos. Matplotlib ofrece una amplia gama de opciones para modificar colores, estilos de línea y tipos de marcadores.

```
plt.plot(x, y, color='red', linestyle='--', marker='o')
```

Estos parámetros no solo cumplen una función estética, sino también funcional, ya que permiten diferenciar múltiples series de datos o destacar tendencias.

Ticks, etiquetas y leyendas

La comprensión de un gráfico depende en gran medida de lo claras que sean sus etiquetas y leyendas. Con xticks() y yticks() es posible modificar las marcas de los ejes, mientras que con legend() se identifican distintas series representadas.

Agregar etiquetas comprensibles y una leyenda bien ubicada mejora significativamente la interpretación del gráfico, especialmente en contextos donde se presentan múltiples variables.

Fijando los límites de un gráfico

Es posible controlar el rango de valores visibles mediante las funciones xlim() y ylim(). Esto es útil para enfocar la atención del lector en un intervalo específico, eliminar valores atípicos o comparar gráficos entre sí con escalas comunes.

Anotaciones y dibujos en un gráfico

Matplotlib permite agregar comentarios o señales visuales dentro del gráfico usando annotate() o text(). Esto puede incluir flechas, recuadros o simples etiquetas que expliquen puntos importantes o anomalías en los datos.

Este tipo de elementos aporta contexto y facilita la comprensión por parte del usuario final.

Guardando los gráficos en un archivo

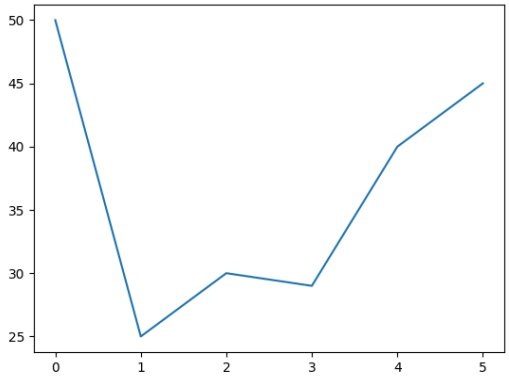
Una vez creado un gráfico, puede guardarse fácilmente con savefig(). Esta función acepta diferentes formatos como PNG, PDF o SVG, además de parámetros como resolución (dpi) y margen (bbox_inches='tight').

Guardar los gráficos permite integrarlos en informes, presentaciones o documentos académicos sin pérdida de calidad.

Tipos de gráficos

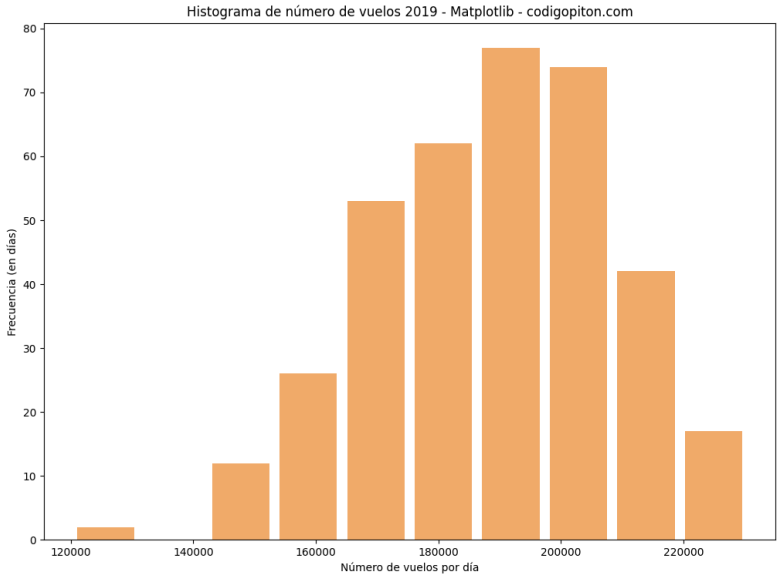
Diagrama de línea

Es el más usado para representar series temporales o secuencias ordenadas. Su forma continua permite visualizar tendencias o variaciones a lo largo del tiempo.



Histograma

Permite ver la distribución de frecuencias de un conjunto de datos. Es útil para detectar asimetrías, acumulaciones o patrones inusuales.



Boxplot

Muestra la dispersión de los datos, incluyendo la mediana, los cuartiles y posibles valores atípicos. Es ideal para análisis comparativos entre grupos.

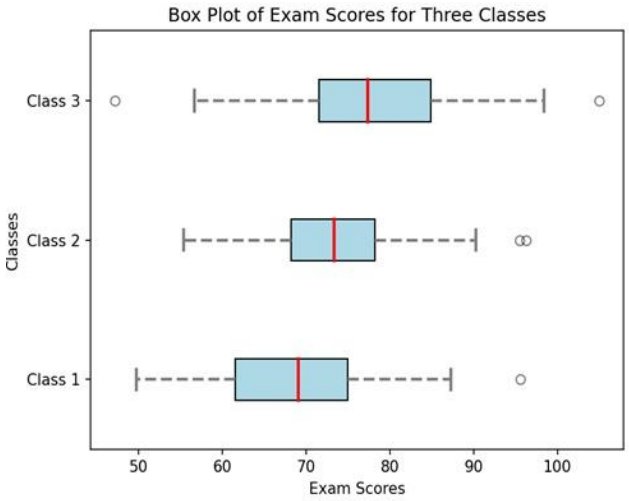


Diagrama de dispersión

Representa dos variables numéricas en un plano cartesiano, permitiendo detectar correlaciones o agrupamientos.

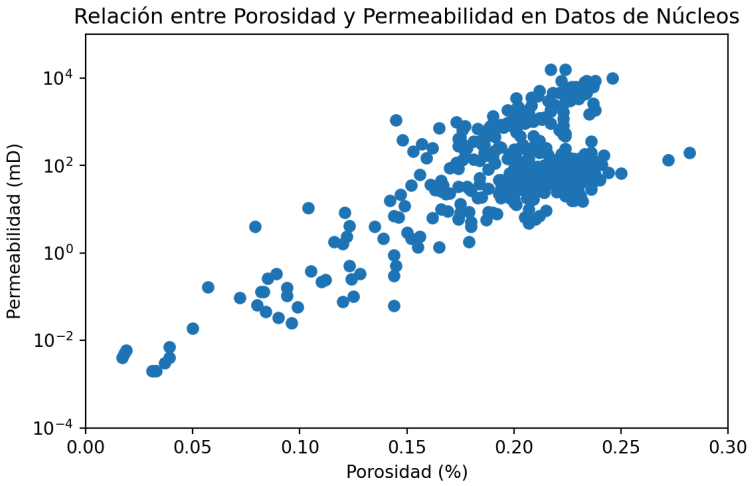


Diagrama de barras

Representa valores categóricos. Es eficaz para comparar cantidades entre distintos grupos.

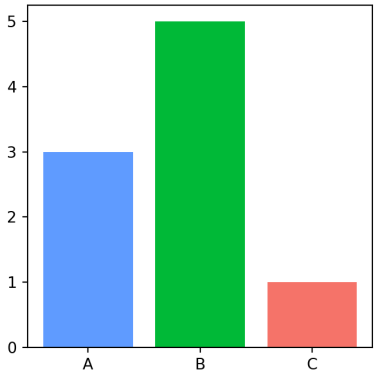
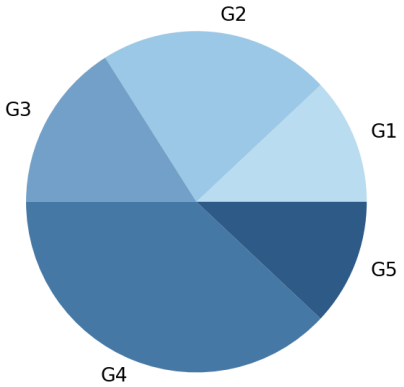


Diagrama de torta

Muestra proporciones relativas de un total. Se recomienda usarlo solo con pocos valores, ya que su legibilidad disminuye con muchas categorías.



Cierre ✍️

El dominio de Matplotlib no solo permite crear gráficos visualmente atractivos, sino también convertir conjuntos de datos complejos en historias visuales claras y significativas. Has aprendido a construir gráficos usando dos modelos distintos, a personalizar cada componente visual, y a elegir el tipo de gráfico adecuado según los datos que se desean representar.

Este conocimiento te habilita a enfrentar con confianza desafíos reales de análisis y visualización de datos, aplicando criterios técnicos y estéticos en cada caso.

¡Nos vemos de nuevo, más adelante! 👋

Referencias📚

- Matplotlib. (2024). *Matplotlib documentation*. https://matplotlib.org/stable/index.html
- VanderPlas, J. (2016). *Python data science handbook*. O'Reilly Media. https://jakevdp.github.io/PythonDataScienceHandbook/
- NumPy Developers. (2024). *NumPy documentation*. https://numpy.org/doc/
- Python Software Foundation. (2024). *Python official documentation*. https://docs.python.org/3/

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy